

Apunte 18 - Autenticación y Autorización

Conceptos fundamentales

Autenticación es el proceso de verificar la identidad de un usuario: confirmar que es quien dice ser. Se basa en uno o más factores:

- **Algo que sabemos:** contraseña, PIN
- **Algo que tenemos:** teléfono (código SMS/OTP), token físico
- **Algo que somos:** huella dactilar, reconocimiento facial

Autorización es el proceso de determinar qué recursos puede acceder un usuario ya autenticado. Que un usuario haya iniciado sesión no significa que pueda acceder a todo el sistema.

Ejemplo concreto: Un usuario con rol "vendedor" puede ver y editar sus artículos, pero no puede administrar usuarios. Un usuario con rol "admin" puede hacer ambas cosas.

Métodos de autenticación

Usuario y contraseña (tradicional)

El método más común. El usuario se registra con credenciales que se almacenan en la base de datos. **La contraseña NUNCA se almacena en texto plano**, sino hasheada.

MFA (Autenticación Multifactor)

Combina dos o más factores de autenticación. Ejemplo: contraseña + código OTP (One Time Passcode) enviado por SMS o generado por una app como Google Authenticator o Authy.

Passwordless (Sin contraseña)

Se envía un enlace mágico (magic link) al correo electrónico del usuario. Al hacer click, inicia sesión sin necesidad de contraseña.

Login por redes sociales (OAuth)

El usuario inicia sesión usando su cuenta de Google, GitHub, Facebook, etc. La aplicación delega la autenticación al proveedor social. Se profundiza más adelante en la sección OAuth 2.0.

Autenticación biométrica

Usa huellas dactilares, reconocimiento facial o de voz. Cada vez más común en dispositivos móviles.

Passkeys (tendencia actual)

Basadas en el estándar **WebAuthn/FIDO2**, las passkeys permiten autenticarse sin contraseña usando criptografía de clave pública. Google, Apple y Microsoft ya las soportan. Representan el futuro de la autenticación web.

Hasdeo seguro de contraseñas

Antes de hablar de JWT, es fundamental entender cómo almacenar contraseñas de forma segura. **Nunca almacenar contraseñas en texto plano ni con hashes débiles** (MD5, SHA-1, SHA-256 sin salt).

bcrypt

El estándar mínimo recomendado. Incluye salt automático y un factor de costo configurable que hace el hash deliberadamente lento (resistente a fuerza bruta).

```
import bcrypt from 'bcryptjs';

// Registrar usuario: hashear la contraseña antes de guardar
async function registrarUsuario(username, password) {
  const salt = await bcrypt.genSalt(12); // factor de costo 12 (recomendado)
  const hashedPassword = await bcrypt.hash(password, salt);

  const usuario = await Usuario.create({
    username,
    password: hashedPassword // se guarda el hash, NUNCA el texto plano
  });
  return usuario;
}

// Login: comparar contraseña ingresada contra el hash almacenado
async function verificarCredenciales(username, passwordIngresada) {
  const usuario = await Usuario.findOne({ where: { username } });
  if (!usuario) return null;

  const esValida = await bcrypt.compare(passwordIngresada, usuario.password);
  if (!esValida) return null;

  return usuario;
}
```

Nota: OWASP recomienda **Argon2id** como primera opción (paquete npm `argon2`), ya que es resistente a ataques con GPU/ASIC. Para nuestro curso, bcrypt es suficiente y más simple de implementar.

JWT (JSON Web Token)

JWT es un estándar abierto (RFC 7519) para transmitir información de forma segura entre partes como un objeto JSON firmado digitalmente. Se usa ampliamente para autenticación y autorización en APIs REST.

Estructura de un JWT

Un JWT consta de tres partes separadas por puntos: `header.payload.signature`

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluaXN0cmFkb3IiLCJpYXQiOiJlE3MDA1MDAwMDB9.firma_digital
```

- **Header:** algoritmo de firma y tipo de token
- **Payload:** datos del usuario (claims). **No incluir información sensible** (contraseñas, tarjetas) porque el payload es decodificable (no encriptado, solo firmado)
- **Signature:** firma digital que garantiza que el token no fue modificado

Instalación

```
npm install jsonwebtoken bcryptjs dotenv
```

Variables de entorno

Crear un archivo `.env` en la raíz del proyecto backend:

```
JWT_SECRET=una_clave_secreta_muy_larga_y_compleja_que_no_se_comparte
JWT_EXPIRES_IN=15m
JWT_REFRESH_SECRET=otra_clave_secreta_diferente_para_refresh
JWT_REFRESH_EXPIRES_IN=7d
```

IMPORTANTE: El archivo `.env` debe estar en `.gitignore`. Nunca subir secretos al repositorio.

Cargar las variables al inicio de la aplicación:

```
import 'dotenv/config';
// Ahora process.env.JWT_SECRET está disponible
```

Implementación paso a paso en Express

1. Modelo de Usuario (Sequelize)

```
import { DataTypes } from 'sequelize';
import { sequelize } from '../config/database.js';

const Usuario = sequelize.define('Usuario', {
  username: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true
  },
},
```

```
password: {
  type: DataTypes.STRING,
  allowNull: false
},
role: {
  type: DataTypes.STRING,
  allowNull: false,
  defaultValue: 'usuario' // roles: 'usuario', 'admin'
}
});

export default Usuario;
```

2. Ruta de registro

```
import bcrypt from 'bcryptjs';
import Usuario from '../models/usuario.js';

app.post('/api/register', async (req, res) => {
  const { username, password } = req.body; // whitelist de campos

  // Verificar si el usuario ya existe
  const existente = await Usuario.findOne({ where: { username } });
  if (existente) {
    return res.status(400).json({ message: 'El usuario ya existe' });
  }

  // Hashear contraseña
  const salt = await bcrypt.genSalt(12);
  const hashedPassword = await bcrypt.hash(password, salt);

  const usuario = await Usuario.create({
    username,
    password: hashedPassword
  });

  res.status(201).json({ message: 'Usuario registrado correctamente' });
});
```

3. Ruta de login (genera access token + refresh token)

```
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';

app.post('/api/login', async (req, res) => {
  const { username, password } = req.body;

  // Buscar usuario
```

```
const usuario = await Usuario.findOne({ where: { username } });
if (!usuario) {
  return res.status(401).json({ message: 'Credenciales inválidas' });
}

// Verificar contraseña
const esValida = await bcrypt.compare(password, usuario.password);
if (!esValida) {
  return res.status(401).json({ message: 'Credenciales inválidas' });
}

// Generar access token (corta duración)
const accessToken = jwt.sign(
  { id: usuario.id, username: usuario.username, role: usuario.role },
  process.env.JWT_SECRET,
  { expiresIn: process.env.JWT_EXPIRES_IN } // 15 minutos
);

// Generar refresh token (larga duración)
const refreshToken = jwt.sign(
  { id: usuario.id },
  process.env.JWT_REFRESH_SECRET,
  { expiresIn: process.env.JWT_REFRESH_EXPIRES_IN } // 7 días
);

// Enviar refresh token como cookie httpOnly
res.cookie('refreshToken', refreshToken, {
  httpOnly: true, // no accesible desde JavaScript (protege contra XSS)
  secure: true, // solo se envía por HTTPS
  sameSite: 'Strict', // protege contra CSRF
  maxAge: 7 * 24 * 60 * 60 * 1000 // 7 días en milisegundos
});

// Enviar access token en el body de la respuesta
res.json({ accessToken });
});
```

¿Por qué dos tokens?

- **Access token** (15 min): se usa en cada petición a la API. Si es robado, el daño es limitado por su corta vida.
- **Refresh token** (7 días): se usa solo para obtener un nuevo access token cuando expira. Se almacena en una cookie httpOnly (inaccesible desde JavaScript).

¿Por qué NO almacenar tokens en localStorage? `localStorage` es accesible desde cualquier script JavaScript en la página. Si hay una vulnerabilidad XSS, un atacante puede leer el token. Las cookies `httpOnly` no son accesibles desde JavaScript, lo que las hace más seguras.

4. Middleware de autenticación (reutilizable)

```
import jwt from 'jsonwebtoken';

function authenticateToken(req, res, next) {
  // El token viene en el header: "Authorization: Bearer <token>"
  const authHeader = req.headers.authorization;
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ message: 'Token no proporcionado' });
  }

  jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
    if (err) {
      return res.status(403).json({ message: 'Token inválido o expirado' });
    }
    req.user = decoded; // { id, username, role }
    next();
  });
}

export default authenticateToken;
```

5. Middleware de autorización por roles

```
function authorizeRole(...rolesPermitidos) {
  return (req, res, next) => {
    if (!rolesPermitidos.includes(req.user.role)) {
      return res.status(403).json({ message: 'No tiene permisos para esta acción' });
    }
    next();
  };
}

export default authorizeRole;
```

Uso combinado de autenticación + autorización:

```
import authenticateToken from './middleware/authenticateToken.js';
import authorizeRole from './middleware/authorizeRole.js';

// Cualquier usuario autenticado puede ver artículos
app.get('/api/articulos', authenticateToken, articulosController.getAll);

// Solo admin puede eliminar artículos
app.delete('/api/articulos/:id',
  authenticateToken,
  authorizeRole('admin'),
```

```
    articulosController.delete
  );

  // Admin y vendedor pueden crear artículos
  app.post('/api/articulos',
    authenticateToken,
    authorizeRole('admin', 'vendedor'),
    articulosController.create
  );
```

6. Ruta de refresh token

```
app.post('/api/refresh', (req, res) => {
  const refreshToken = req.cookies.refreshToken;

  if (!refreshToken) {
    return res.status(401).json({ message: 'Refresh token no encontrado' });
  }

  jwt.verify(refreshToken, process.env.JWT_REFRESH_SECRET, async (err, decoded) => {
    if (err) {
      return res.status(403).json({ message: 'Refresh token inválido' });
    }

    // Buscar usuario actualizado (por si cambió de rol, fue eliminado, etc.)
    const usuario = await Usuario.findByPk(decoded.id);
    if (!usuario) {
      return res.status(403).json({ message: 'Usuario no encontrado' });
    }

    // Generar nuevo access token
    const nuevoAccessToken = jwt.sign(
      { id: usuario.id, username: usuario.username, role: usuario.role },
      process.env.JWT_SECRET,
      { expiresIn: process.env.JWT_EXPIRES_IN }
    );

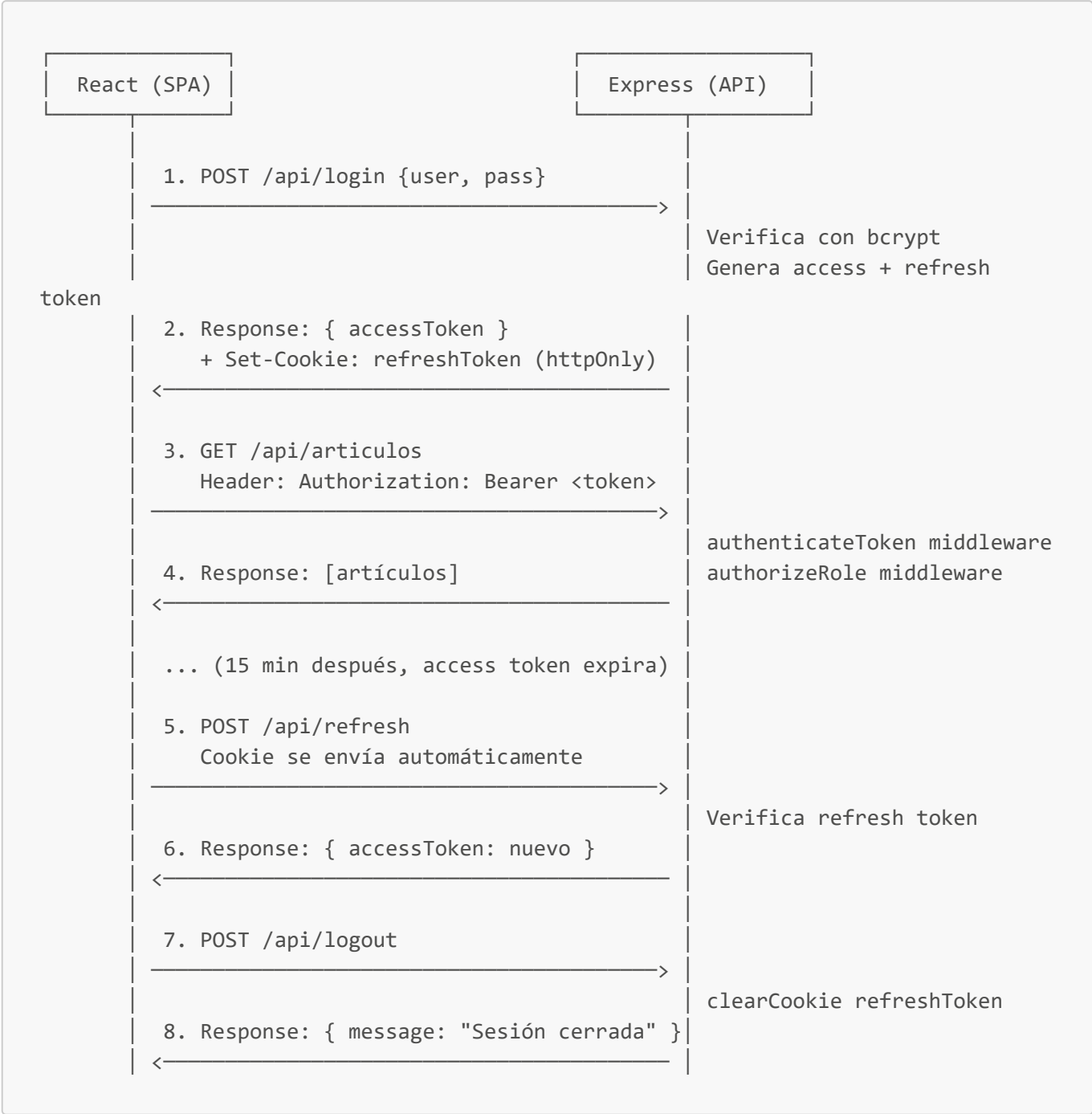
    res.json({ accessToken: nuevoAccessToken });
  });
});
```

7. Ruta de logout

```
app.post('/api/logout', (req, res) => {
  res.clearCookie('refreshToken', {
    httpOnly: true,
    secure: true,
```

```
    sameSite: 'Strict'
  });
  res.json({ message: 'Sesión cerrada' });
});
```

Flujo completo de autenticación



OAuth 2.0

OAuth 2.0 es un protocolo de autorización que permite a una aplicación acceder a recursos de un usuario en otro servicio, sin que el usuario comparta sus credenciales directamente.

Ejemplo cotidiano: "Iniciar sesión con Google" — nuestra app no recibe la contraseña de Google del usuario, sino un token que autoriza el acceso a ciertos datos (nombre, email, foto).

Flujo simplificado:

1. El usuario hace click en "Iniciar sesión con Google"
2. Se redirige al servidor de Google (Authorization Server)
3. El usuario autoriza a nuestra app
4. Google redirige de vuelta a nuestra app con un código de autorización
5. Nuestro backend intercambia ese código por un access token directamente con Google
6. Con ese token, podemos obtener los datos del usuario desde la API de Google

OAuth 2.1 (draft actual) consolida las mejores prácticas: PKCE obligatorio para todos los clientes, flujo implícito deprecado.

Configuración necesaria en Express para cookies

Para que las cookies funcionen correctamente con el patrón de refresh token, necesitamos `cookie-parser`:

```
npm install cookie-parser
```

```
import express from 'express';
import cookieParser from 'cookie-parser';

const app = express();
app.use(express.json());
app.use(cookieParser());
```

Resumen de dependencias de seguridad

```
npm install jsonwebtoken bcryptjs dotenv cookie-parser helmet cors express-rate-limit
```

Paquete	Propósito
<code>jsonwebtoken</code>	Generar y verificar tokens JWT
<code>bcryptjs</code>	Hashear y verificar contraseñas
<code>dotenv</code>	Cargar variables de entorno desde <code>.env</code>
<code>cookie-parser</code>	Leer cookies en Express (para refresh token)
<code>helmet</code>	Cabeceras de seguridad HTTP

Paquete	Propósito
<code>cors</code>	Configurar Cross-Origin Resource Sharing
<code>express-rate-limit</code>	Limitar intentos de login/registro

Ejercicios

Ejercicio Seguridad 1: Implementar registro y login con hasheo de contraseñas.

1. Crear el modelo `Usuario` en Sequelize con campos `username`, `password` y `role`
2. Implementar `POST /api/register` que hashee la contraseña con `bcrypt` antes de guardar
3. Implementar `POST /api/login` que verifique credenciales con `bcrypt.compare` y devuelva un access token JWT
4. Probar con Postman: registrar un usuario, hacer login y verificar que el token se genera correctamente

Ejercicio Seguridad 2: Proteger rutas con middleware de autenticación.

1. Crear el middleware `authenticateToken` como se muestra en el apunte
2. Proteger las rutas de artículos: GET puede ser público, pero POST/PUT/DELETE requieren autenticación
3. Probar con Postman: verificar que sin token se recibe 401, y con token válido se accede correctamente
4. Probar con un token expirado o inválido y verificar que se recibe 403

Ejercicio Seguridad 3: Implementar autorización por roles.

1. Crear el middleware `authorizeRole` como se muestra en el apunte
2. Crear al menos dos usuarios con roles diferentes (admin y usuario)
3. Configurar que solo admin pueda eliminar artículos
4. Probar con Postman: loguearse como usuario normal e intentar eliminar (debe recibir 403), loguearse como admin y verificar que puede eliminar

Ejercicio Seguridad 4 (avanzado): Implementar refresh tokens.

1. Agregar `cookie-parser` y configurar la ruta `POST /api/login` para enviar el refresh token como cookie httpOnly
2. Implementar `POST /api/refresh` que genere un nuevo access token a partir del refresh token de la cookie
3. Implementar `POST /api/logout` que limpie la cookie del refresh token
4. Probar el flujo completo con Postman: login → usar access token → refresh → logout